



Socket Select



Reti di Calcolatori Modulo 2

Select → Orchestra + server software ≠ parallelismo

- ▶ È una system call che permette di gestire più socket contemporanei
- ▶ Il programma si blocca fino a quando uno tra i descrittori dei socket non è pronto per la lettura o scrittura
- ▶ Quando uno o più socket sono pronti la select ritorna l'elenco dei descrittori pronti

Select

- ▶ Identifica i socket che sono “pronti per l’uso”, su cui sono disponibili dei dati
 - ▶ Può essere bloccante permanentemente, bloccante per un tempo limitato o non bloccante
 - ▶ Input: una serie di descrittori di file
 - ▶ Output: informazioni sullo stato dei descrittori di file

Select

- ▶ `int select(int maxfd, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, struct timeval *timeout);`
- ▶ La `select()` controlla quale tra i primi `maxfd` descrittori
 - ▶ Si rende pronto per lettura (`readfds`)
 - ▶ Si rende pronto per scrittura (`writefds`)
 - ▶ Risulta soggetto a condizioni eccezionali (`exceptfds`)
- ▶ Il tipo `fd_set` è un vettore di bit, ciascuno associato a un descrittore
- ▶ Dopo la chiamata
 - ▶ La `select` ritorna il numero di descrittori pronti per I/O (int)
 - ▶ Il vettore di bit è modificato per mostrare i descrittori pronti per I/O

Select

- ▶ Gli fdset sono inizializzati con
 - ▶ `FD_ZERO(fd_set *fdset); /* clear all bits */`
 - ▶ `FD_SET(int fd, fd_set *fdset); /* turn on a bit */`
 - ▶ `FD_CLR(int fd, fd_set *fdset); /* turn off a bit */`
- ▶ Gli fdset sono testati con
 - ▶ `FD_ISSET(int fd, fd_set *fdset); /* test a bit */`

Select: Timeout

```
struct timeval {  
    long tv_sec; /* seconds */  
    long tv_usec; /* microseconds */  
}
```

- ▶ Timeout vale null, select bloccante
- ▶ Timeout punta a struct timeval, si blocca per timeval
- ▶ Se i campi di timeval sono 0, polling

Select: Tipologie

- ▶ Esistono due tipologie di select
 - ▶ Select per la gestione di socket “server” (creati a partire dalla chiamata a funzione `socket()`)
 - ▶ Select per la gestione di socket “client” (creati a partire dalla chiamata a funzione `accept()`)

Tipologia 1 – Socket Select Server

- ▶ Gestione di servizi (socket) multipli con un unico server (processo)
- ▶ Server
 - ▶ Crea una socket per ogni servizio
 - ▶ Chiama la funzione *select* che aspetta una richiesta
 - ▶ *Select* specifica quale servizio deve essere contattato
- ▶ Select si blocca e monitora un insieme di socket descriptor, fino a quando uno non diventa pronto per essere servito

Tipologia 1 – Socket Select Server

- ▶ Il server si pone in attesa di due socket (server) pronti per la lettura

```
void main() {  
    int sfd1, sfd2; → Socket description  
    fd_set readfds;  
    int n;  
    struct timeval tv;  
    int maxfd;  
  
    sfd1 = socket(AF_INET, SOCK_DGRAM, 0);  
    sfd2 = socket(AF_INET, SOCK_DGRAM, 0);  
  
    bind(sfd1,...);  
    bind(sfd2,...);  
  
    //Inizializzo la var readfds  
    FD_ZERO(&readfds);  
    FD_SET(sfd1, &readfds);  
    FD_SET(sfd2, &readfds);
```

Tipologia 1 – Socket Select Server

```
//Inizializzazione timeout 3 secondi
tv.tv_sec = 3;
tv.tv_usec = 0;
//max file descriptor
maxfd = sfd1;
maxfd = (maxfd > sfd2 ? maxfd : sfd2);

n = select(maxfd + 1, &readfds, NULL, NULL, &tv);

if (n > 0) {
    if (FD_ISSET(sfd1, &readfds)) {
        ... //recvfrom
    }
    if (FD_ISSET(sfd2, &readfds)) {
        ... //recvfrom
    }
}
...
close(sfd1);
close(sfd2);
}
```



Tipologia 1 – Socket Select Server

▶ Si supponga che

▶ sockfd1=5

▶ sockfd2=7

▶ sockfd2 diventa pronto per la **lettura**

Tipologia 1 – Socket Select Server

▶ FD_ZERO(&readfs)

	0	1	2	3	4	5	6	7	8	9
readfs	0	0	0	0	0	0	0	0	0	0

▶ FD_SET(5,&readfs) e FD_SET(7,&readfs)

	0	1	2	3	4	5	6	7	8	9
readfs	0	0	0	0	0	1	0	1	0	0

▶ Subito dopo la select(8,&readfs,null,null,null)

	0	1	2	3	4	5	6	7	8	9
readfs	0	0	0	0	0	0	0	1	0	0

Esercizio 1

- ▶ Scrivere un software distribuito (server+client) nel linguaggio C, con **socket TCP select**
- ▶ Il software deve gestire due server socket
 - ▶ Il server UDP echo dato come template in ariel
 - ▶ Il server che TCP iterativo che fa la concatenazione di stringhe implementato nei precedenti esercizi
- ▶ Commentare opportunamente il codice, ponendo in evidenza le parti significative di un server socket select

Esercizio 2

- ▶ Scrivere un software distribuito (server+client) nel linguaggio C, con **socket TCP select**
- ▶ Il software deve estendere il software all'esercizio 1 con un socket che fornisce un servizio somma di due interi. Utilizzare le funzioni `hton*` e `ntoh*`
- ▶ Commentare opportunamente il codice, ponendo in evidenza le parti significative di un server socket select

Tipologia 2 – Socket Select Client

- ▶ Uso di select per gestire un server e i suoi client
 - ▶ Ad ogni richiesta, se la richiesta è sul socket server crea nuove connessioni client (slave)
 - ▶ Ogni connessione è inserita in un fd_set dopo aver fatto l'accept
 - ▶ Se la richiesta è su un socket client
 - ▶ Il servizio implementato viene eseguito e il risultato viene ritornato al client attraverso la socket
- ▶ Esempio echo server
 - ▶ Quando un dato è disponibile in uno o più socket, il server riceve il dato e lo rimanda indietro sullo stesso socket tramite un'echo

Tipologia 2 – Socket Select Client

```
fd_set readset, tempset;
int maxfd, flags;
int srvsock, peersock, j, result, result1, sent, len;
timeval tv;
char buffer[MAX_BUFFER_SIZE+1];
sockaddr_in addr;
/* codice per creare il socket associarlo ad una porta e chiamare la listen
    srvsock = socket(...);
    bind(srvsock ...);
    listen(srvsock ...);
*/
FD_ZERO(&readset);
FD_SET(srvsock, &readset);
maxfd = srvsock;
```



Tipologia 2 – Socket Select Client

```
do {
    memcpy(&tempset, &readset, sizeof(readset));
    tv.tv_sec = 30;
    tv.tv_usec = 0;
    result = select(maxfd + 1, &tempset, NULL, NULL, &tv);
    if (result == 0) {
        printf("select() timed out!\n");
    }
    else if (result < 0 && errno != EINTR) {printf("Error in select(): %s\n", strerror(errno));}
    else if (result > 0) {
        if (FD_ISSET(srvsock, &tempset)) {
            len = sizeof(addr);
            peersock = accept(srvsock, &addr, &len);
            if (peersock < 0) {printf("Error in accept(): %s\n", strerror(errno));}
            else {
                FD_SET(peersock, &readset);
                maxfd = (maxfd < peersock)?peersock:maxfd;
            }
        }
        FD_CLR(srvsock, &tempset);
    }
}
```



Tipologia 2 – Socket Select Client

```
for (j=0; j<maxfd+1; j++) {  
    if (FD_ISSET(j, &tempset)) {  
        do {  
            result = recv(j, buffer, MAX_BUFFER_SIZE, 0);  
        } while (result == -1 && errno == EINTR);  
  
        if (result > 0) {  
            buffer[result] = '\0'; //null terminate  
            printf("Echoing: %s\n", buffer);  
            sent = 0;  
  
            do {  
                result1 = send(j, buffer+sent, result-sent, MSG_NOSIGNAL);  
                if (result1 > 0)  
                    sent += result1;  
                else if (result1 < 0 && errno != EINTR);  
                break;  
            } while (result > sent);  
        }  
    }
```

Tipologia 2 – Socket Select Client

```
else if (result == 0) {
    close(j);
    FD_CLR(j, &readset);
}
else {
    printf("Error in recv(): %s\n", strerror(errno));
}
} // end if (FD_ISSET(j, &tempset))
} // end for (j=0;...)
} // end else if (result > 0)
} while (1);
```

Esercizio 3

- ▶ Scrivere un software distribuito (server+client) nel linguaggio C, con **socket TCP select**
- ▶ Il software deve gestire un server socket a vostra scelta (echo server o in alternativa la concatenazione di stringhe) in parallelo a tutte le socket slave che gestiscono i singoli client
- ▶ Commentare opportunamente il codice, ponendo in evidenza le parti significative di un server socket select